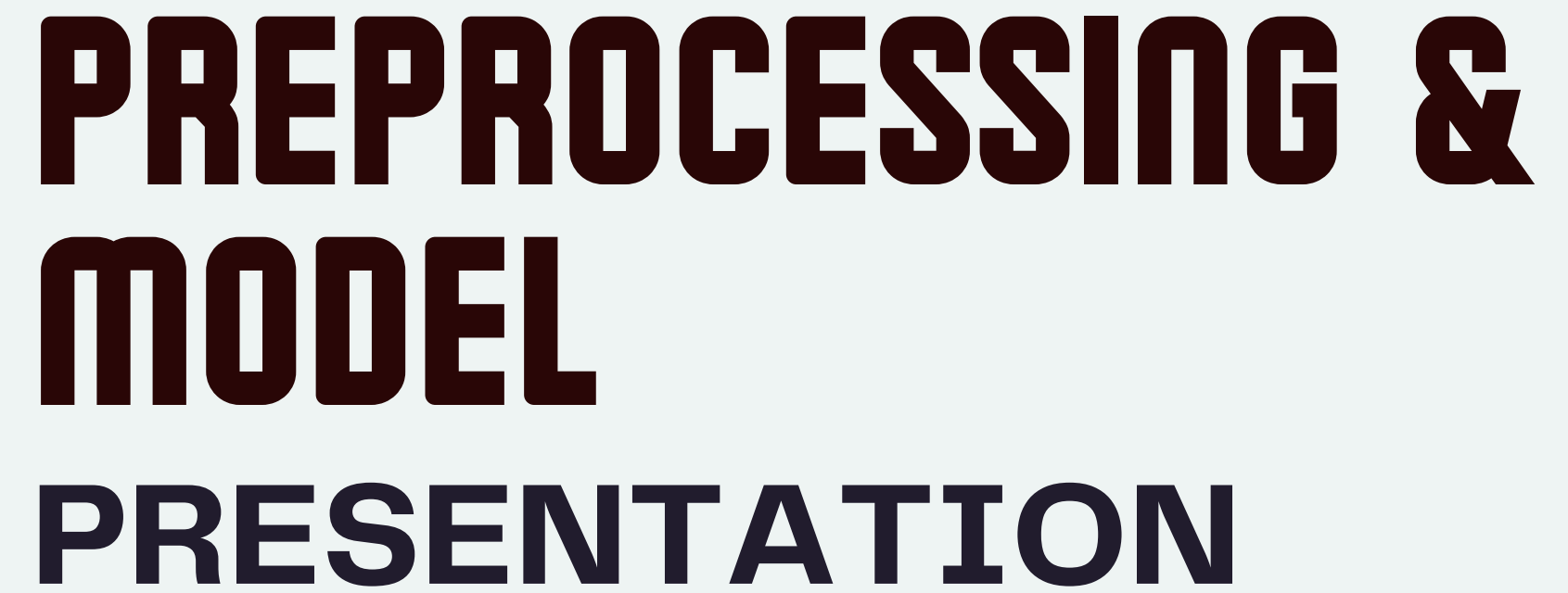
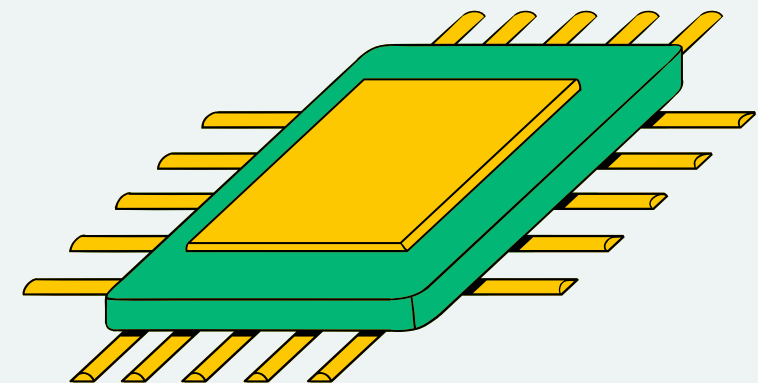




ALYA HAMIDAH





PRESENTATION OUTLINE

- Preprocessing
- Feature Engineering
- Model Selection
- Training



PREPROCESSING DATA



Binary Encoding

One-Hot Encoding

Ordinal Encoding

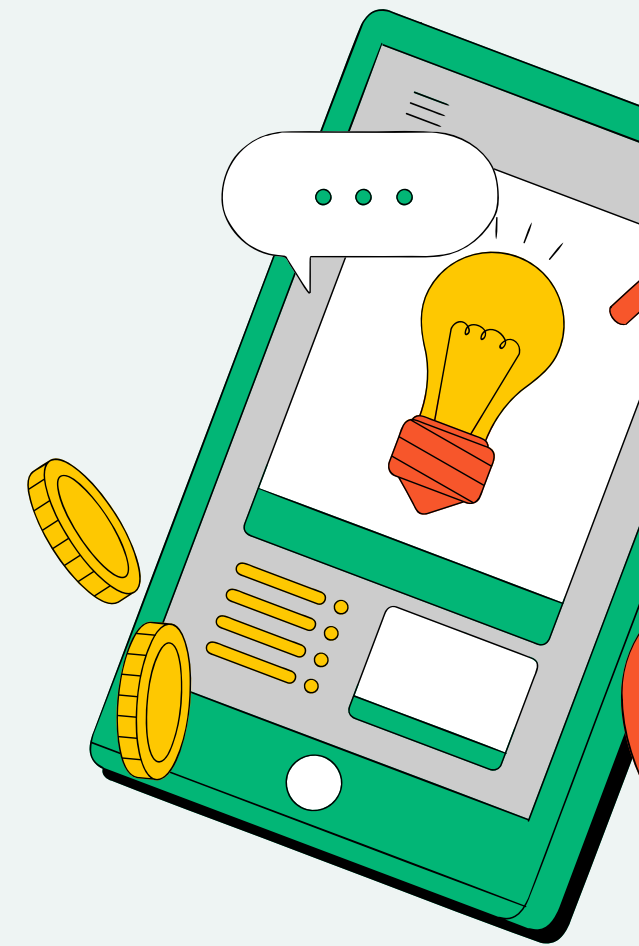
Binning

Feature Scalling (Min-Max Scalling)

Undersampling



PREPROCESSING DATA - BINARY ENCODING



```
[ ] # Membuat kolom baru dengan nilai boolean berdasarkan kolom asli
sdd_df['Suicidal Thoughts (Boolean)'] = sdd_df['Have you ever had suicidal thoughts ?'].map({'Yes': True, 'No': False})

# Menampilkan hasil perubahan
print(sdd_df[['Have you ever had suicidal thoughts ?', 'Suicidal Thoughts (Boolean)']].head())
```

	Have you ever had suicidal thoughts ?	Suicidal Thoughts (Boolean)
0	Yes	True
1	No	False
2	No	False
3	Yes	True
4	Yes	True

```
[ ] # Mengubah 'Yes' menjadi True dan 'No' menjadi False
sdd_df['Family History of Mental Illness (Boolean)'] = sdd_df['Family History of Mental Illness'].map({'Yes': True, 'No': False})

# Menampilkan hasil perubahan
print(sdd_df[['Family History of Mental Illness (Boolean)', 'Family History of Mental Illness']].head())
```

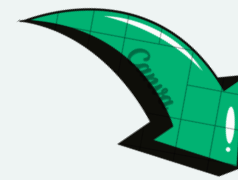
	Family History of Mental Illness (Boolean)	Family History of Mental Illness
0	False	No
1	True	Yes
2	True	Yes
3	True	Yes
4	False	No



PREPROCESSING DATA - ONE HOT ENCODING

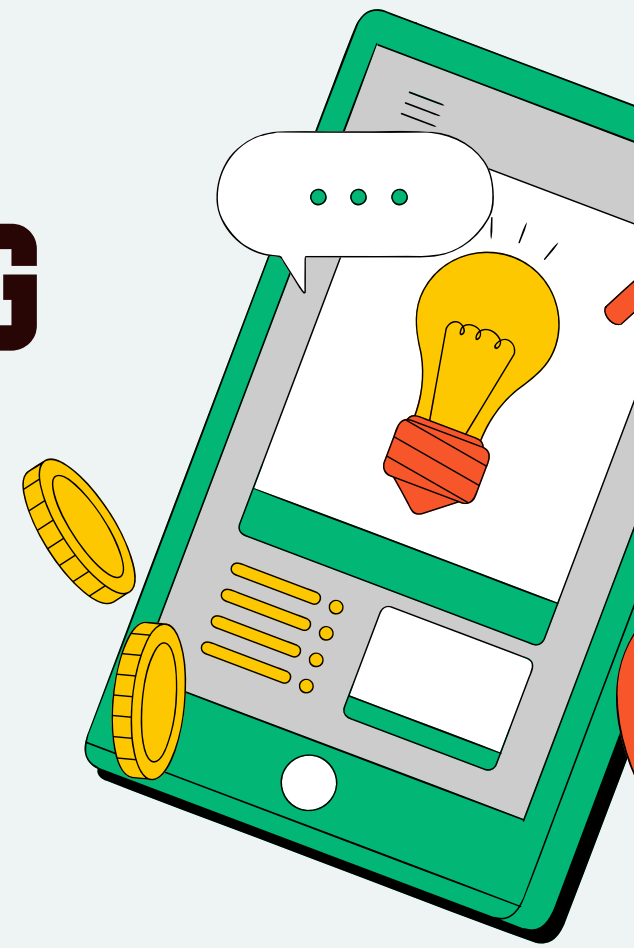
```
print(sdd_df['gender'].head())
```

```
0    Male
1  Female
2    Male
3  Female
4  Female
Name: gender, dtype: object
```



```
gender_one = pd.get_dummies(sdd_df, columns=['gender'])
print(gender_one[['gender_Female', 'gender_Male']].head())
```

```
gender_Female  gender_Male
0           False         True
1            True         False
2           False         True
3            True         False
4            True         False
```



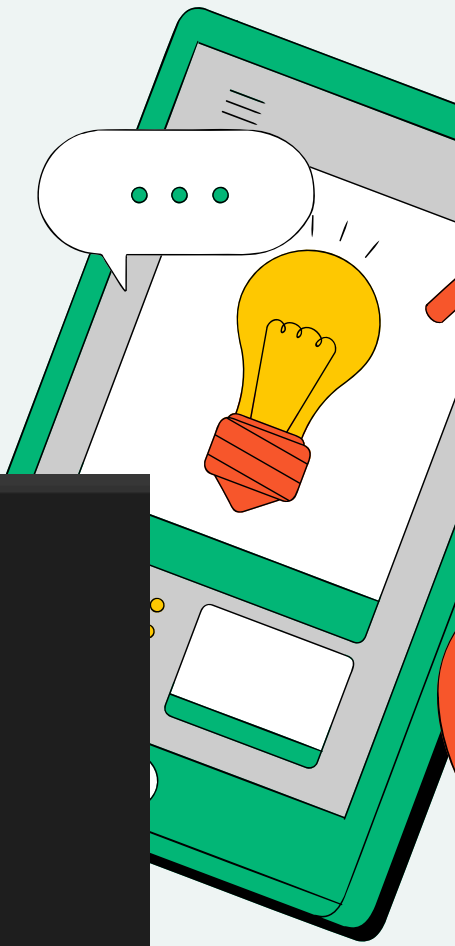
PREPROCESSING DATA - ORDINAL ENCODING

```
[4] dietary_mapping = {  
    'Unhealthy': 1,  
    'Moderate': 2,  
    'Healthy': 3  
}  
  
sdd_df['dietary_habits_encoded'] = sdd_df['dietary_habits'].map(dietary_mapping)  
  
print(sdd_df[['dietary_habits', 'dietary_habits_encoded']].head())
```

	dietary_habits	dietary_habits_encoded
0	Healthy	3
1	Moderate	2
2	Healthy	3
3	Moderate	2
4	Moderate	2

```
def s_berapa(x):  
    if x=='Class 12':  
        return 0  
    elif x[0] == 'B':  
        return 1  
    elif x[0] == 'M':  
        return 2  
    elif x=='LLM':  
        return 2  
    elif x=='LLB':  
        return 1  
    else:  
        return 0  
  
sdd_df['nomor_degree'] = sdd_df['degree'].apply(s_berapa)  
print(sdd_df[['degree', 'nomor_degree']].head())
```

	degree	nomor_degree
0	B.Pharm	1
1	BSc	1
2	BA	1
3	BCA	1
4	M.Tech	2



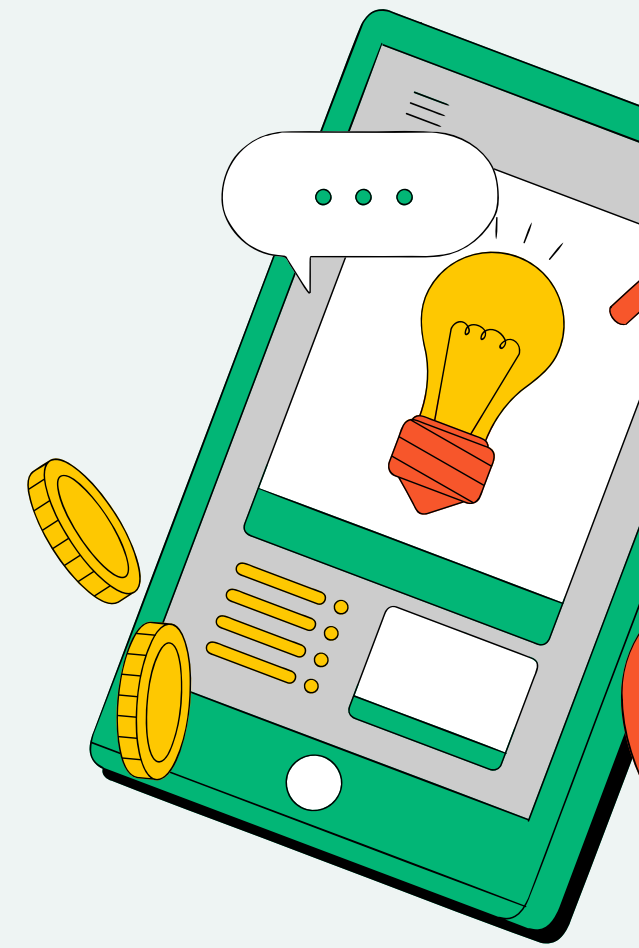
PREPROCESSING DATA - BINNING

```
[16] # Binning umur dengan rentang 18-34 dan 35-59
      bins = [18, 34, 59]
      labels = ['18-34', '35-59']
      sdd['kategori_umur'] = pd.cut(sdd['age'], bins=bins, labels=labels, right=True, include_lowest=True)
```

```
[17] # Binning umur dengan rentang 18-27, 28-37, 38-59
      bins = [18, 27, 37, 59]
      labels = ['18-27', '28-37', '38-59']
      sdd['kategori_umur_2'] = pd.cut(sdd['age'], bins=bins, labels=labels, right=True, include_lowest=True)
```

```
print(sdd[['age', 'kategori_umur', 'kategori_umur_2']].tail(10))
```

	age	kategori_umur	kategori_umur_2
27760	28.0	18-34	28-37
27761	20.0	18-34	18-27
27762	24.0	18-34	18-27
27763	23.0	18-34	18-27
27764	31.0	18-34	28-37
27765	27.0	18-34	18-27
27766	27.0	18-34	18-27
27767	31.0	18-34	28-37
27768	18.0	18-34	18-27
27769	27.0	18-34	18-27



PREPROCESSING DATA - FEATURE SCALING

```
from sklearn.preprocessing import MinMaxScaler

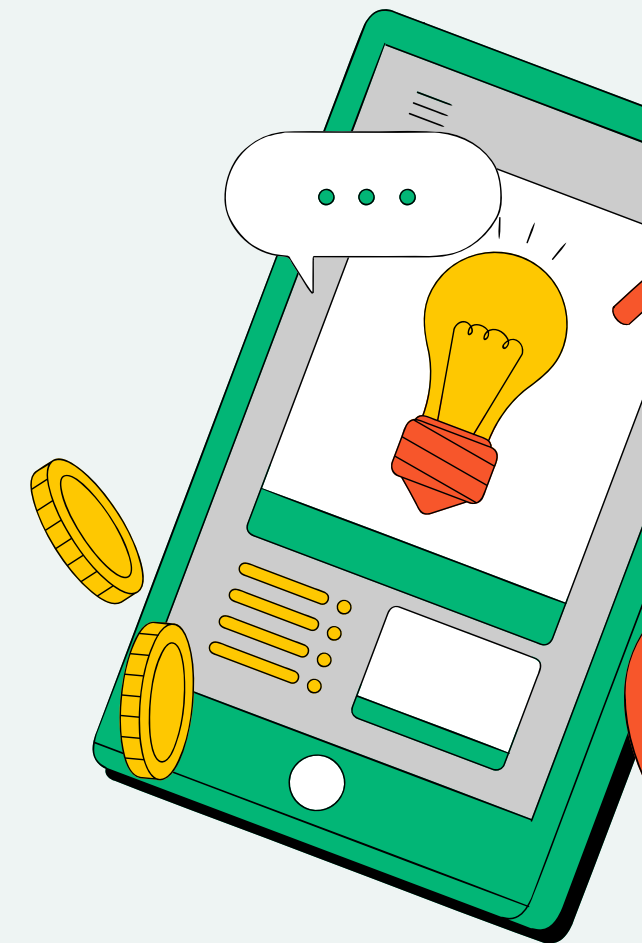
scaler = MinMaxScaler(feature_range=(0, 1))
df_scaled = pd.DataFrame(scaler.fit_transform(df), columns=df.columns)
print(df_scaled.head())
```

```
↔
```

	age	academic_pressure	cgpa	study_satisfaction	\
0	0.365854	1.0	0.792757	0.4	
1	0.146341	0.4	0.175050	1.0	
2	0.317073	0.6	0.402414	1.0	
3	0.243902	0.6	0.112676	0.4	
4	0.170732	0.8	0.623742	0.6	

	work_study_hours	financial_stress	depression	is_student	\
0	0.250000	0.00	1.0	0.0	
1	0.250000	0.25	0.0	0.0	
2	0.750000	0.00	0.0	0.0	
3	0.333333	1.00	1.0	0.0	
4	0.083333	0.00	0.0	0.0	

	sleep_duration_encoded	suicidal_thoughts_bool	family_history_bool	\
0	0.333333	1.0	0.0	
1	0.333333	0.0	1.0	
2	0.000000	0.0	1.0	
3	0.666667	1.0	1.0	
4	0.333333	1.0	0.0	



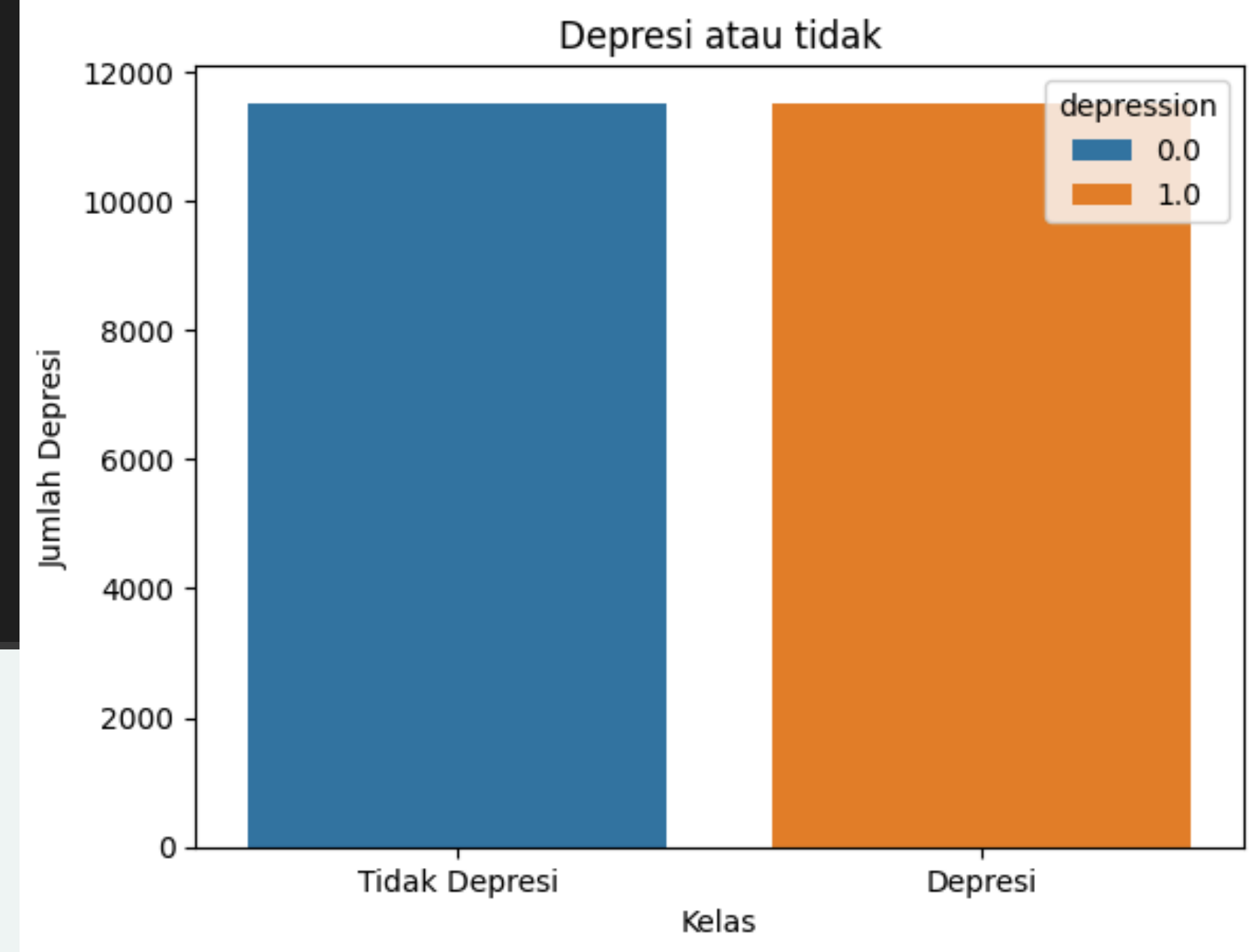
PREPROCESSING DATA - UNDERSAMPLING

```
[24] X = df_scaled.drop(['depression'], axis=1)
      y = df_scaled['depression']

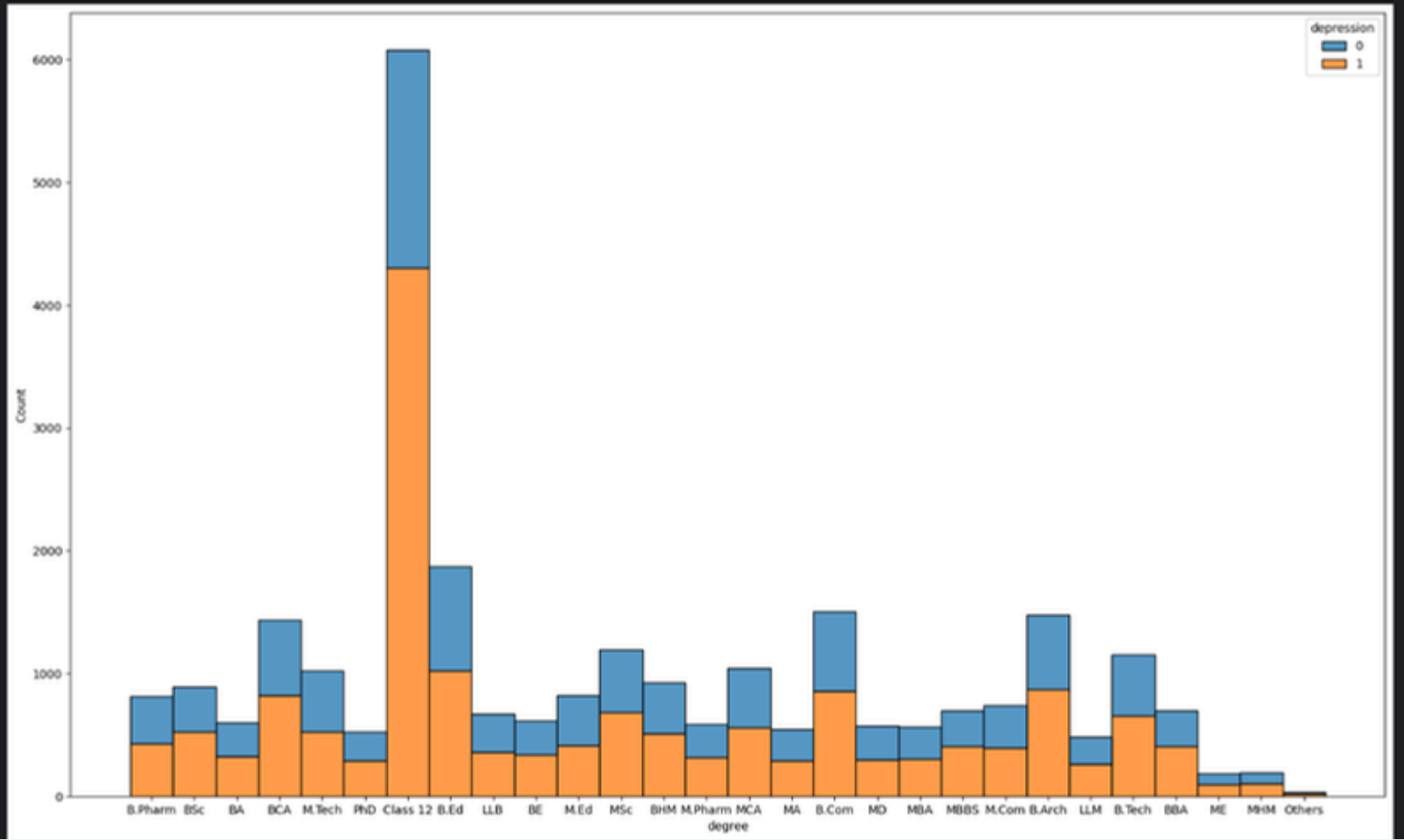
[25] import seaborn as sns
      from imblearn.under_sampling import RandomUnderSampler
      import matplotlib.pyplot as plt

      undersampler = RandomUnderSampler(random_state=42)
      X_resampled, y_resampled = undersampler.fit_resample(X, y)

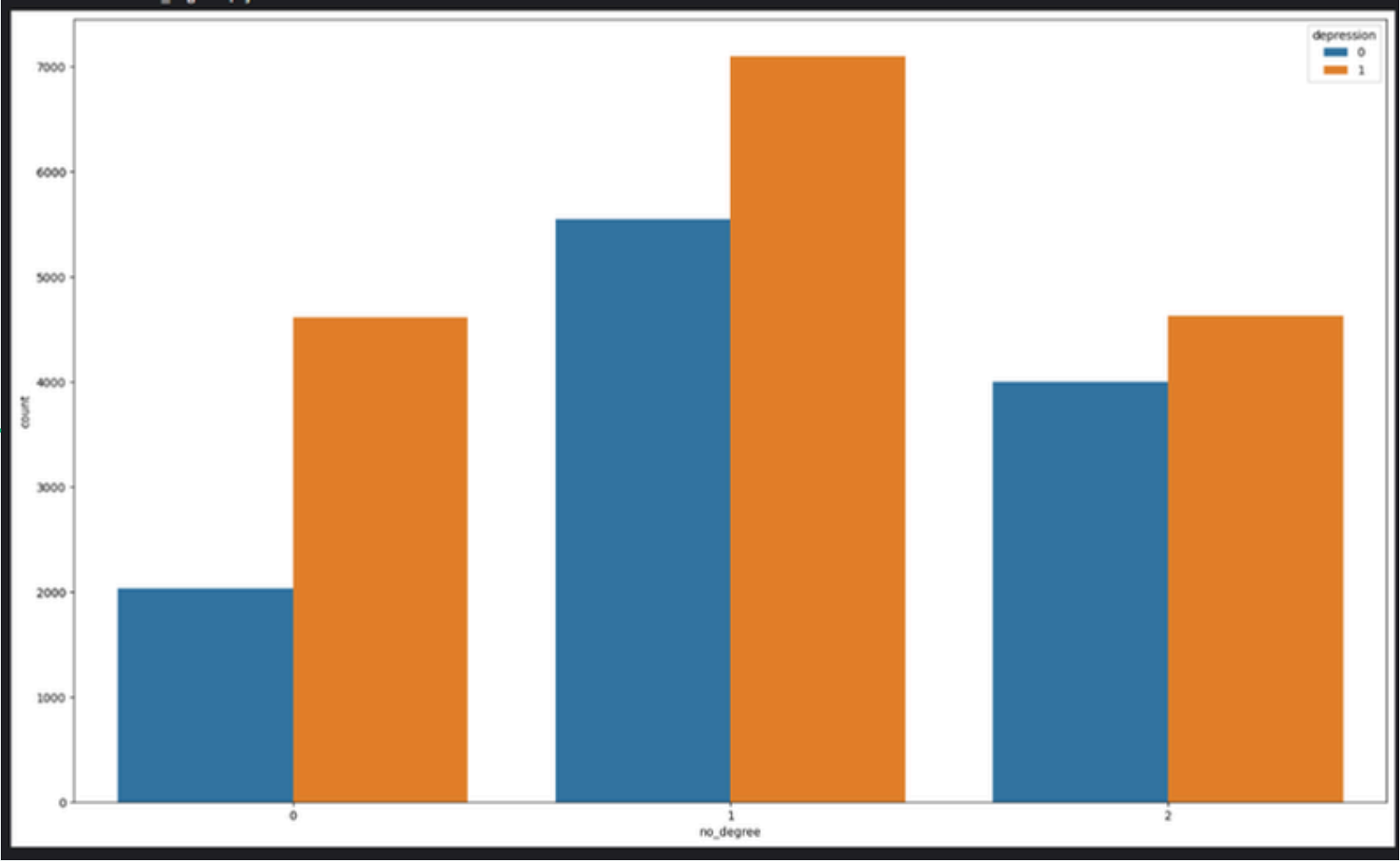
      sns.countplot(x=y_resampled, hue=y_resampled)
      plt.title("Depresi atau tidak")
      plt.xlabel("Kelas")
      plt.ylabel("Jumlah Depresi")
      plt.xticks([0, 1], ['Tidak Depresi', 'Depresi'])
      plt.show()
```



FEATURE ENGINEERING #1



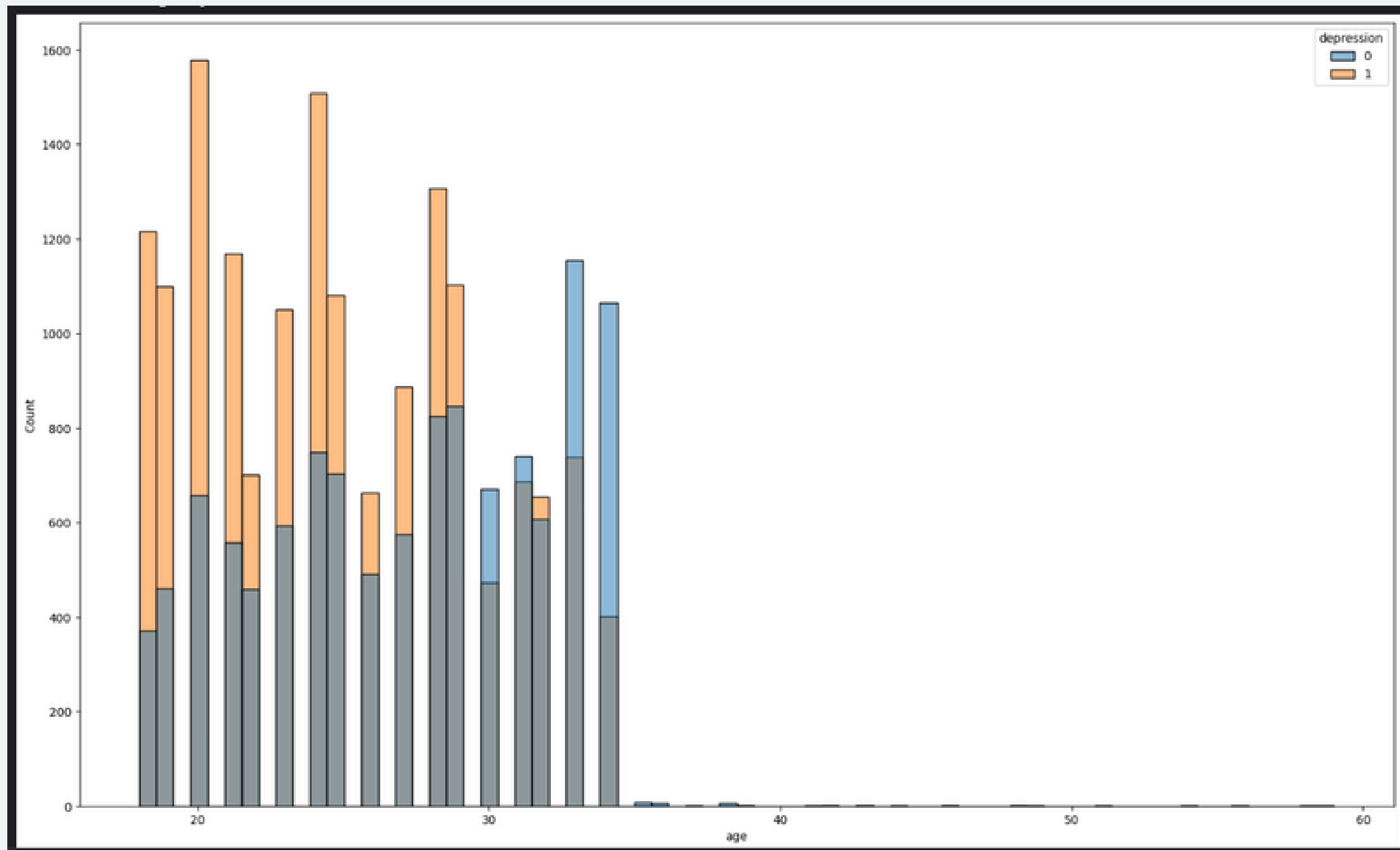
degree



nomor_degree



FEATURE ENGINEERING #2



- binning
- membuat batasan dari umur 34



METODE FEATURE SELECTION

CHI-SQUARE

MUTUAL INFORMATION

F-SCORE

score tertinggi



MODEL

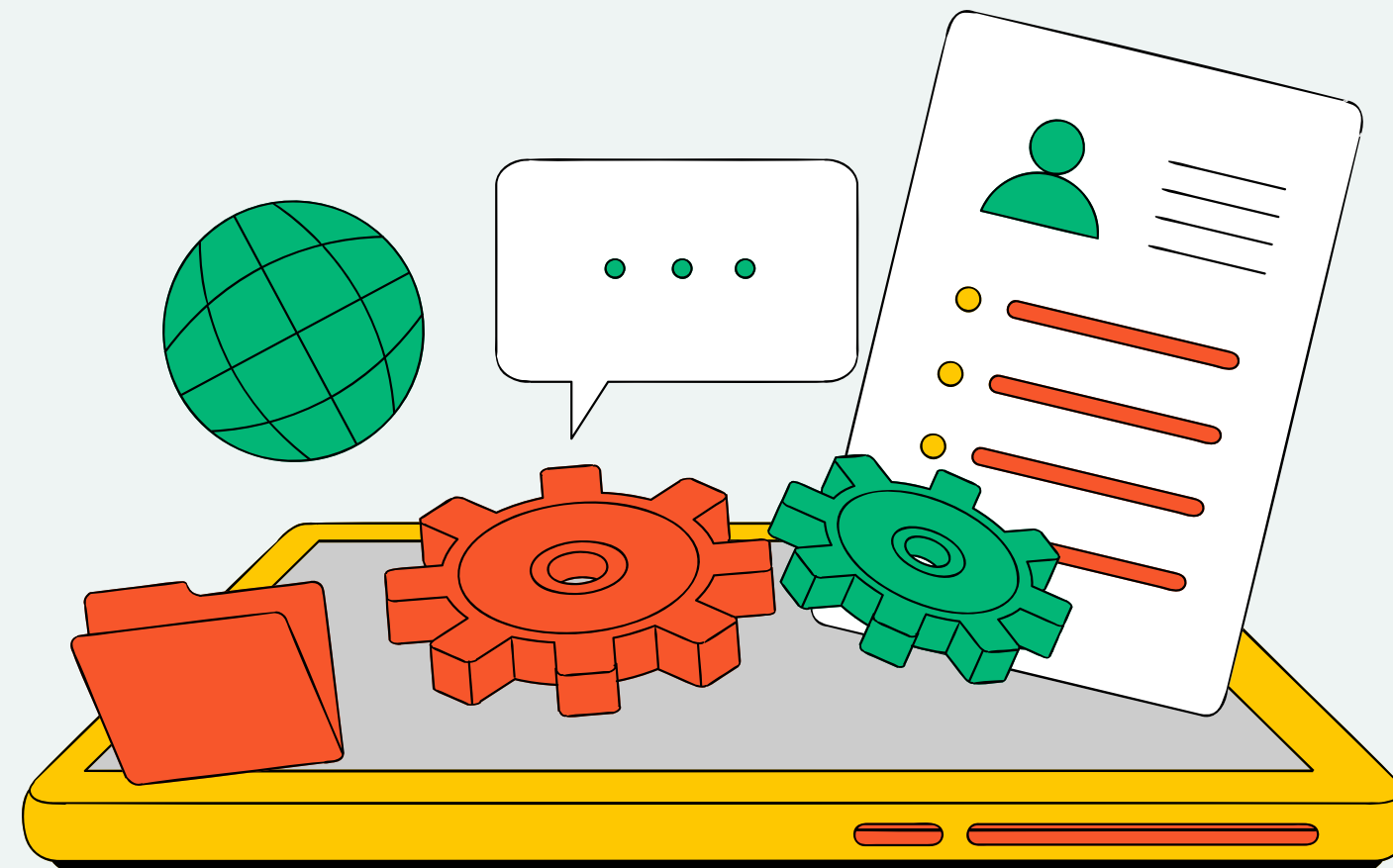
SCORE TERTINGGI

Logistic Regression

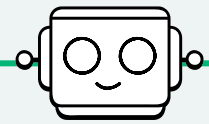
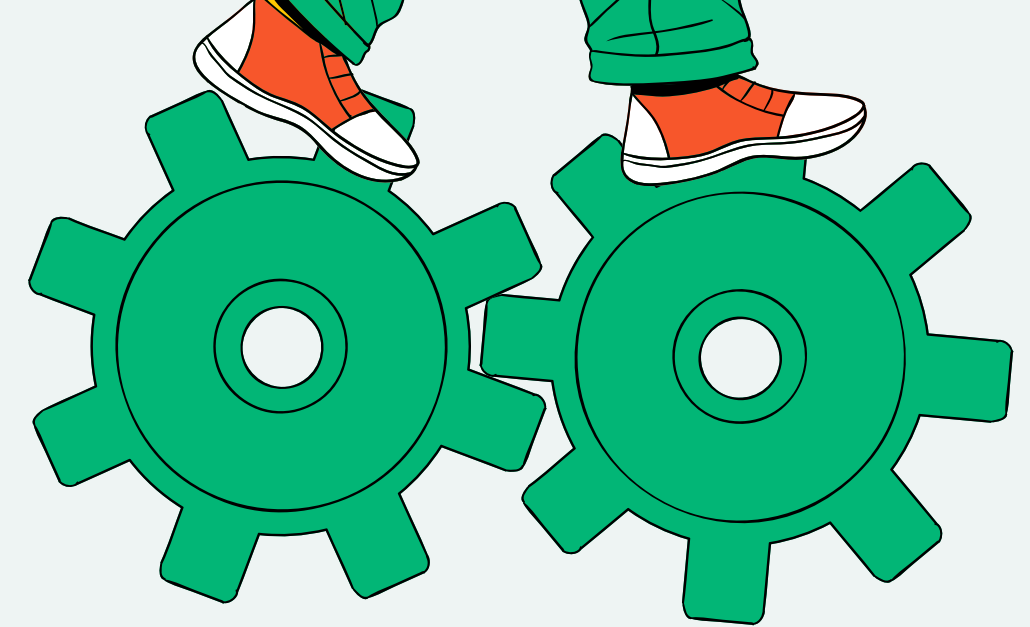
Random Forest

Support Vector Machine (SVM)

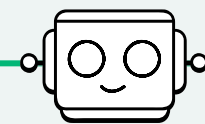
XGBoost



TRAINING



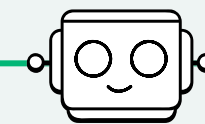
Melakukan training menggunakan data yang di-undersampled dan yang tidak dengan keempat algoritma



Melihat perbedaan hasil pelatihan model dengan fitur hasil feature selection dan dengan keseluruhan fitur



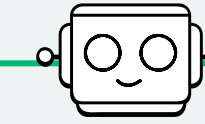
Pelatihan dengan keseluruhan fitur dataset memiliki score lebih tinggi



Melakukan hyperparameter tuning untuk model logistic regression dan xgboost



Menggunakan RandomizedSearchCV

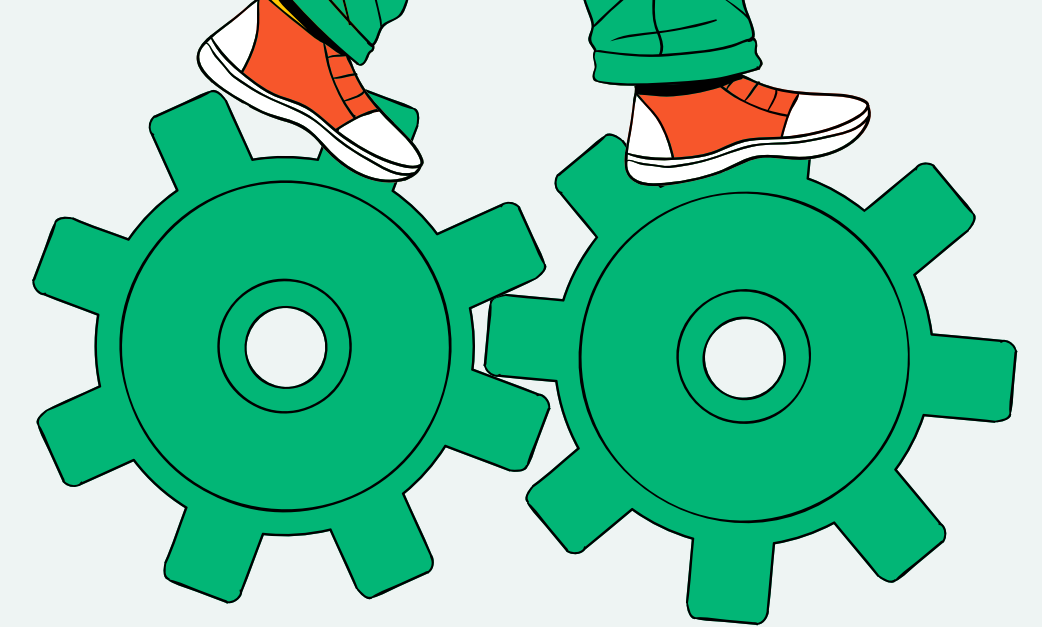


Next



HASIL TRAINING #1

MENGGUNAKAN SEMUA FITUR



LogisticRegression

Evaluation:

recall_score: 0.8929012345679013
f1_score: 0.8729631864815933
accuracy_score: 0.848397551314368
precision_score: 0.8538961038961039

SVC

Evaluation:

recall_score: 0.8950617283950617
f1_score: 0.8728367193378479
accuracy_score: 0.8478574000720202
precision_score: 0.8516886930983847

RandomForestClassifier

Evaluation:

recall_score: 0.8716049382716049
f1_score: 0.8607131971959767
accuracy_score: 0.8354339214980194
precision_score: 0.8500903070439494

XGBClassifier

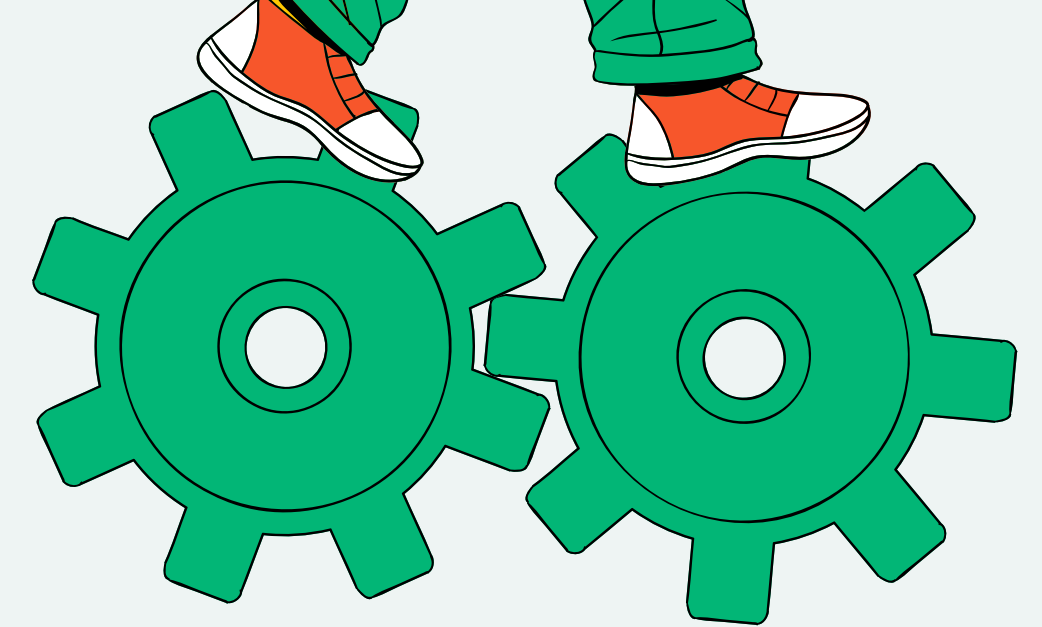
Evaluation:

recall_score: 0.8737654320987654
f1_score: 0.8614027080480756
accuracy_score: 0.8359740727403673
precision_score: 0.8493849384938494



HASIL TRAINING #2

MENGGUNAKAN SEMUA FITUR-UNDERSAMPLED



LogisticRegression

Evaluation:

recall_score: 0.8555893866898652
f1_score: 0.8460215053763441
accuracy_score: 0.8446180555555556
precision_score: 0.836665248830285

SVC

Evaluation:

recall_score: 0.8499347542409743
f1_score: 0.8420599008834303
accuracy_score: 0.8409288194444444
precision_score: 0.8343296327924851

RandomForestClassifier

Evaluation:

recall_score: 0.841670291431057
f1_score: 0.8353118929419384
accuracy_score: 0.8344184027777778
precision_score: 0.8290488431876607

XGBClassifier

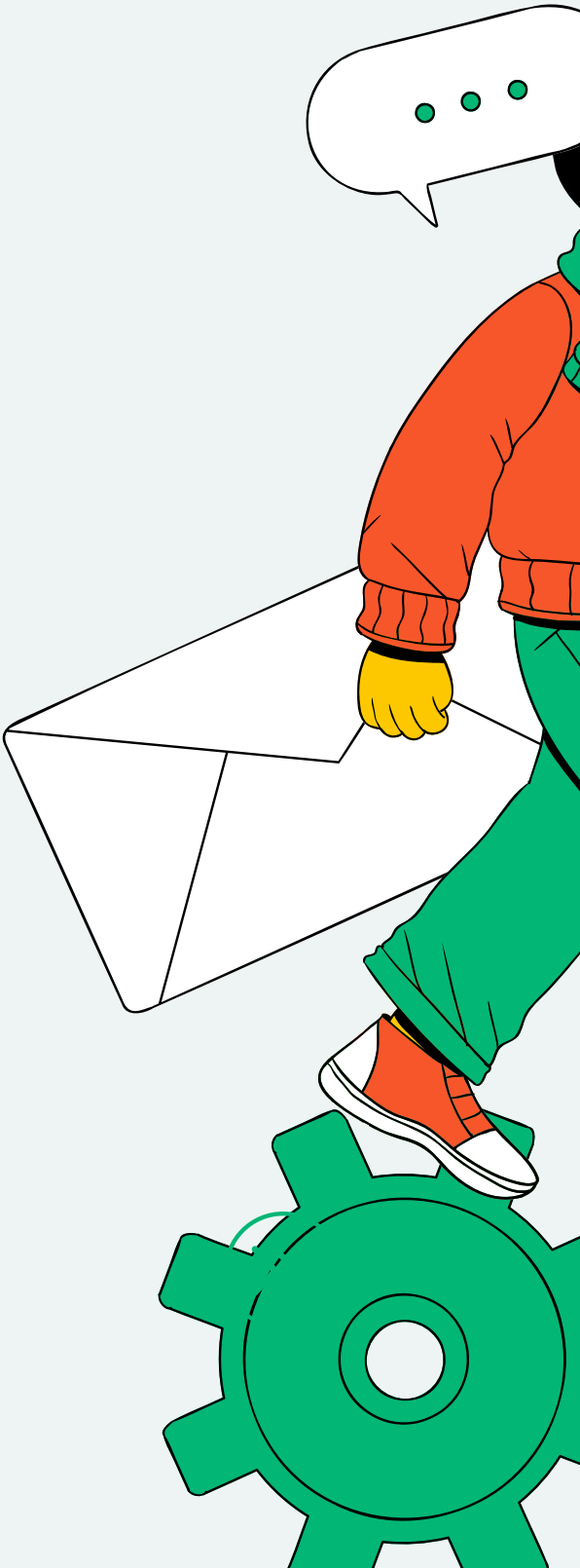
Evaluation:

recall_score: 0.8412353197042193
f1_score: 0.8312916398022782
accuracy_score: 0.8296440972222222
precision_score: 0.8215802888700084



HASIL TRAINING #3 MENGGUNAKAN FITUR DARI FEATURE SELECTION

		f-score	Mutual Info	Chi-Squared
LR	Undersampled	85.55%	85.29%	85.55%
	Tidak	88.95%	88.64%	88.95%
SVM	Undersampled	85.34%	85.29%	85.34%
	Tidak	88.45%	88.36%	88.45%
RF	Undersampled	82.51%	83.21%	83.29%
	Tidak	85.40%	85.24%	86.00%
XGB	Undersampled	83.60%	83.60%	83.60%
	Tidak	86.88%	86.88%	86.88%

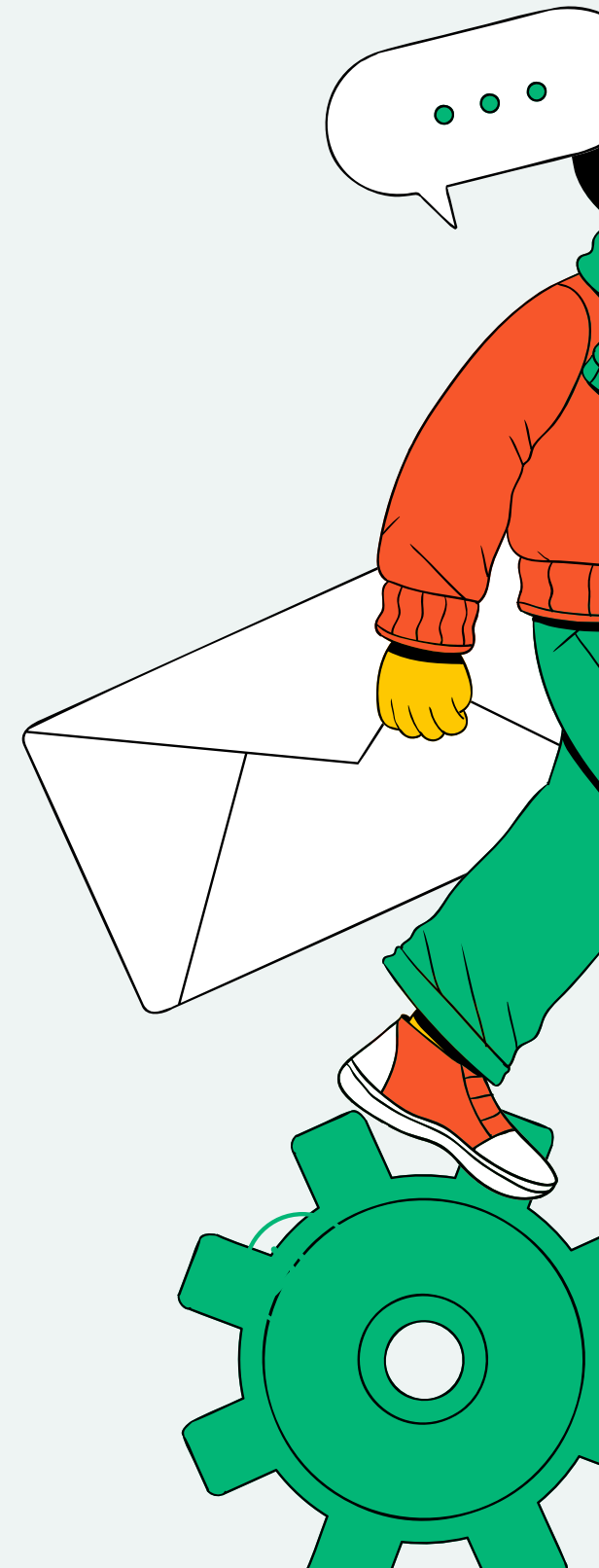


HASIL TRAINING #4 HYPERPARAMETER TUNING

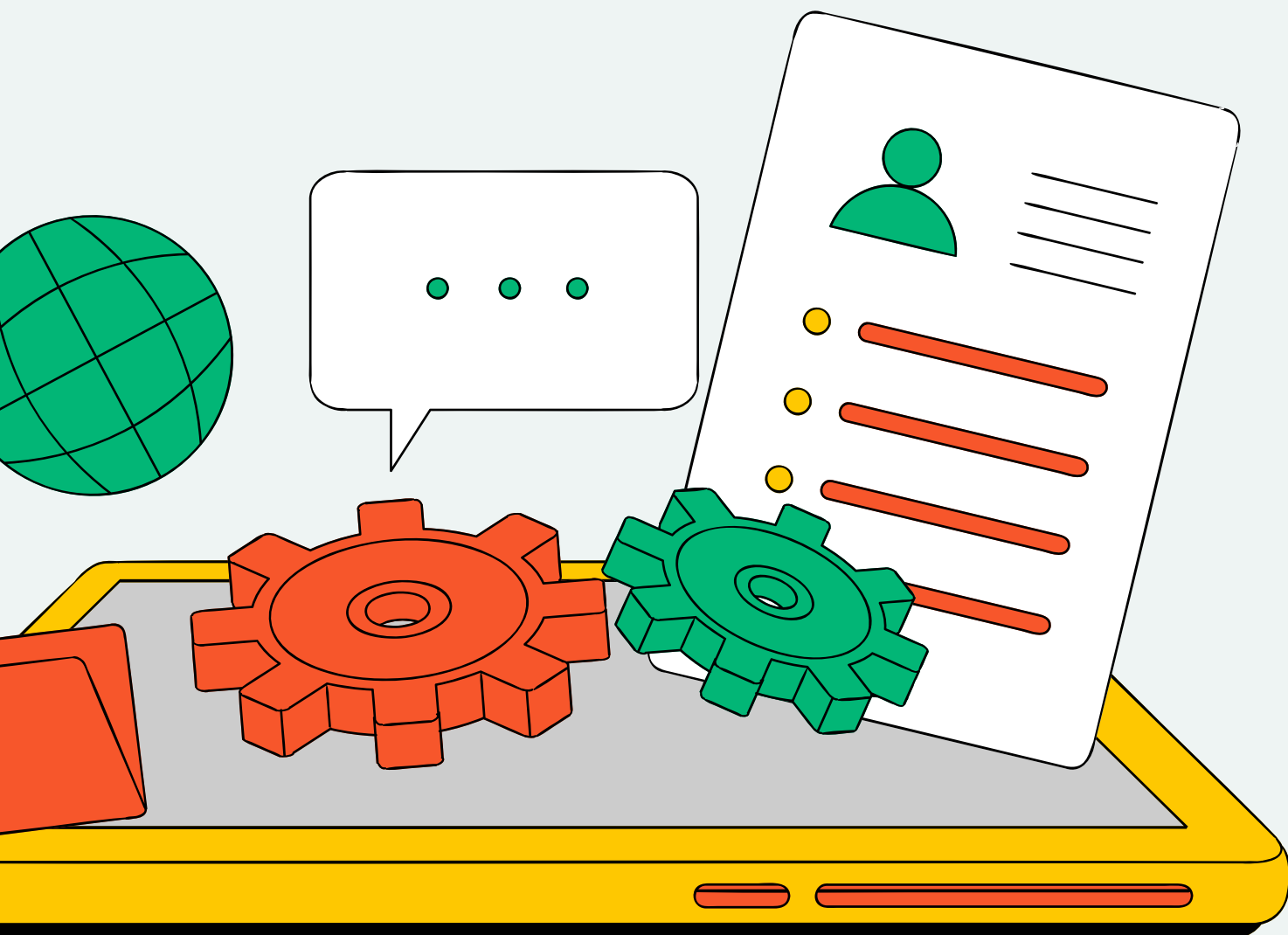
Logistic Regression: 89.41%

XGBoost: 88.67%

**RANDOMIZED SEARCH
CV**



KESIMPULAN



Model terbaik: Logistic Regression

Untuk kedepannya:
Dapat digunakan berbagai metode untuk meningkatkan akurasi/skor evaluasi model, misalnya ensemble, feature engineering kembali, hyperparameter tuning lebih baik, dll



TERIMA KASIH

